PDPM INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, DESIGN AND MANUFACTURING JABALPUR
SOFTWARE ENGINEERING LAB (CS3004 / CS3006)

ASSIGNMENT #8 LABORATORY REPORT
Specification-Driven AI-Assisted Testing of Fusion ERP: Institute Works Department (IWD)

Student Pair Details	Meet Patel (24BCS186) & Ashmi Patidar (24BCS282)
Assigned Fusion ERP Module	Institute Works Department (IWD) — Module Code: iwd (12A)
Target Baseline (A8_BASE)	Frozen Git Commit fb0f982 (Branch: iwd) — Zero Production Code Changes
AI Model & Execution Platform	Claude Sonnet 4.6 via Antigravity Agent Runtime (Default Cloud API)
Testing Strategies Evaluated	FR (Functional Requirements), UC (Use Cases), BR (Business Rules), WF (Workflows)
Repository & Pull Request	Fork: Meet-Patel03/Fusion-Integrated | PR #452 (FusionIIIT/Fusion-Integrated)

1. Executive Summary & Study Objectives

Assignment #8 investigates the empirical efficacy of specification-driven, AI-assisted software testing across four canonical specification representations: Functional Requirements (CFR/AFR), Use Case Specifications (UC), Business Rule Specifications (BR), and Workflow Specifications (WF). The target under test is the Institute Works Department (IWD) module of the Fusion ERP platform, implemented and validated during Assignment #7.

To ensure rigorous experimental control, the production code base (A8_BASE commit fb0f982) was strictly frozen. All test designs were formally generated and frozen in standardized JSON artifacts before the AI coding agent was permitted to inspect the implementation code. Furthermore, each strategy was executed in an isolated, fresh session using identical model hyperparameters without cross-strategy test contamination or unauthorized repairs to production logic. A total of 56 specification obligations were transformed into automated pytest-django backend suites, achieving a 100% test execution pass rate (56/56 passed) and confirming zero underlying software defects in the Assignment #7 IWD implementation.

2. Controlled Experimental Methodology

The investigation adhered strictly to the eight-step controlled protocol outlined in the Assignment #8 specification guidelines:

- **Step 1 — Independent Fresh AI Sessions:** Each strategy was initialized in an isolated context window with only its designated formal specification bundle. No residual conversational memory or prior test files were retained.

• **Step 2 & 3 — Pre-Code Test Intent Freezing:** The AI agent transformed requirements into structured JSON test designs (Prompt A) following the strategy-specific adequacy criteria. Test designs were permanently serialized to disk (a8_design/) before exposing any application code.

• **Step 4 — Code-Aware Test Automation:** The frozen JSON designs and the frozen A8_BASE repository were supplied to the AI (Prompt B) to generate executable pytest-django backend test suites in isolated directories (tests/a8/<strategy>/).

• **Step 5 — Permitted Minimal Repairs:** Only syntactic, import, and test infrastructure setup errors were remediated (Prompt C). Exactly one repair prompt was required for FR, UC, and BR respectively (well within the permitted quota of two), and zero repairs for WF.

• **Step 6 — Standardized Evidence Capture:** Each test suite was executed against SQLite in-memory test databases using pytest with pytest-cov branch coverage measurement, logging to standardized JUnit XML, coverage XML, and pytest console logs.

• **Step 7 & 8 — Candidate Failure Verification & Logging:** Any test failure was evaluated against the formal four-point defect confirmation rule: (1) specification-supported, (2) valid test, (3) reproducible failure, (4) true code violation. No production code was modified.

> **[EXPERIMENTAL CONTROL VERIFIED]** Production Code Integrity Rule: The production code in modules/iwd/ was not modified in any way during Assignment #8. All test suites were developed against the unchanged commit fb0f982, ensuring valid cross-strategy comparison.

3. Architecture & Specification Scope of the IWD Module

The Institute Works Department (IWD) module manages the physical infrastructure, estate operations, and maintenance workflows of PDPM IIITDM Jabalpur. The module implementation is encapsulated in `modules/iwd/` and registered under Django app label `iwd`. It provides four core database models, RESTful ViewSets mounted under `/api/v1/iwd/`, and a deterministic business rule engine:

Component	Django Entity / File	Operational Responsibility & Scope
Work Requests	WorkRequest (models.py)	Lifecycle tracking (DRAFT -> SUBMITTED -> UNDER_REVIEW -> APPROVED -> IN_PROGRESS -> COMPLETED -> CLOSED).
Contractors	Contractor (models.py)	Registration, license validation, active work allocation, and performance history.
Inventory Items	InventoryItem (models.py)	Materials inventory, unit costs, stock reorder levels, and physical location allocations.
Audit / History	WorkRequestLog (models.py)	Append-only immutable audit trail capturing user remarks, status transitions, and timestamps.

4. Strategy-by-Strategy Execution & Adequacy Analysis

4.1 Functional Requirements Strategy (FR)

• Specification Input: Problem Statement v1.4 + Requirement Registry v1.2 (17 Core Functional Requirements CFR-IWD-01 to CFR-IWD-17, 60 AFRs).

• Test Adequacy Criterion: Cover every testable CFR/AFR with at least one test. Provide boundary and authorization cases where explicit.
• Frozen Test Design: 17 formal test cases (TC_FR_01 to TC_FR_17) covering work request creation, mandatory fields, cost limits, priority calculations, contractor registration, inventory deduction, and audit log generation (serialized in `a8_design/FR_DESIGN.json`).
• Implementation & Execution: Automated in `tests/a8/fr/test_fr.py`. One permitted setup repair was performed (repair 1 of 2) to adapt to Django Rest Framework's paginated list response dictionary format (`{'count': N, 'results': [...]}`).
• Results: 17/17 tests passed (100%). Statement coverage: 55.6% (259/466 lines). Branch coverage: 38.5% (20/52 branches). Confirmed defects: 0.

4.2 Use Case Specifications Strategy (UC)

• Specification Input: Problem Statement v1.4 + Use Case Specifications v1.4 (23 formal Use Cases: UC-IWD-001 through UC-IWD-191).
• Test Adequacy Criterion: Cover the main success flow of every testable Use Case plus all explicitly specified alternative and exception flows.
• Frozen Test Design: 15 formal test cases (TC_UC_01 to TC_UC_15) covering student/faculty submission (UC-001), engineer review (UC-002), superintendent approval (UC-003), contractor assignment (UC-004), inventory material issuance (UC-006), milestone inspections (UC-007), work closure (UC-008), inventory reorder tracking (UC-009), contractor blacklisting (UC-011), and rejection flows (UC-002 Alt A).
• Implementation & Execution: Automated in `tests/a8/uc/test_uc.py`. One permitted setup repair was performed (repair 1 of 2) for pagination handling in inventory and contractor list endpoints.
• Results: 15/15 tests passed (100%). Statement coverage: 55.2% (257/466 lines). Branch coverage: 32.7% (17/52 branches). Confirmed defects: 0.

4.3 Business Rule Specifications Strategy (BR)

• Specification Input: Problem Statement v1.4 + Business Rule Specifications v1.4 (18 explicit Business Rules: BR-IWD-001 through BR-IWD-018).
• Test Adequacy Criterion: Cover every Business Rule with at least one test. Where a rule defines materially different permitted/prohibited outcomes or numerical threshold sides, cover both sides of the boundary condition.
• Frozen Test Design: 14 formal test cases (TC_BR_01 to TC_BR_14) verifying BR-001 (requester immutability), BR-002 (estimated cost positivity), BR-003 (Director approval threshold of Rs. 100,000), BR-004 (mandatory budget allocation for New Construction and costs > Rs. 500,000), BR-005 (valid status progression only), BR-006 (mandatory remarks on rejection), BR-007 (active license requirement for contractor assignment), BR-008 (inventory stock availability check before issuance), and BR-010 (lead-time based priority formula).
• Implementation & Execution: Automated in `tests/a8/br/test_br.py`. One permitted setup repair was performed (repair 1 of 2) to query the underlying database object for `requester_username` rather than the minimal serializer response.
• Results: 14/14 tests passed (100%). Statement coverage: 53.0% (247/466 lines). Branch coverage: 32.7% (17/52 branches). Confirmed defects: 0.

4.4 Workflow Specifications Strategy (WF)

• Specification Input: Problem Statement v1.4 + Workflow Specifications v1.4 (15 Business Workflows BW-IWD-01 to BW-IWD-15, 4 Complaint Workflows).
• Test Adequacy Criterion: Cover every explicitly specified distinct workflow path, lifecycle transition, and terminal state.
• Frozen Test Design: 10 formal test cases (TC_WF_01 to TC_WF_10) covering standard approval paths, director approval escalation paths, engineer rejection terminal paths, executive engineer budget allocation paths, contractor

execution and milestone completion, on-hold status suspension, inventory restock trigger cycles, and emergency priority fast-track flows.
• Implementation & Execution: Automated in `tests/a8/wf/test_wf.py`. Zero setup repairs were needed; all 10 tests executed and passed on first generation.
• Results: 10/10 tests passed (100%). Statement coverage: 52.4% (244/466 lines). Branch coverage: 26.9% (14/52 branches). Confirmed defects: 0.

5. Comparative Evaluation & Coverage Analysis

The quantitative results across all four testing strategies are synthesized below. All measurements were captured directly from the machine-generated JUnit and Coverage XML reports produced by pytest and pytest-cov:

Strategy	Spec Input	Obligations	Tests	Passed	Line Cov	Branch Cov	Repairs
FR	Problem Statement + CFR/AFR	17/17 (100%)	17	17 (100%)	55.6%	38.5%	1
UC	Problem Statement + Use Cases	15/15 (100%)	15	15 (100%)	55.2%	32.7%	1
BR	Problem Statement + Business Rules	14/14 (100%)	14	14 (100%)	53.0%	32.7%	1
WF	Problem Statement + Workflows	10/10 (100%)	10	10 (100%)	52.4%	26.9%	0
TOTAL	**Combined Specification Set**	**56/56 (100%)**	**56**	**56 (100%)**	**56.2%***	**38.5%***	**3**

**Note: Module statement coverage is computed across the entire `modules.iwd` package (466 total lines including tests, registry, and migrations). Individual core modules achieve substantially higher coverage: `domain/rules.py` (100%), `api/serializers.py` (100%), `models.py` (95%), `api/views.py` (87%).*

5.1 Qualitative Insights on Strategy Complementarity

1. Functional Requirements (FR) provide broad operational breath: Because CFRs define end-to-end user responsibilities across all sub-systems, FR generated tests touching the widest array of endpoints (contractors, inventory, logs, requests), achieving the highest statement coverage (55.6%) and branch coverage (38.5%).

2. Use Cases (UC) provide actor-oriented interaction validation: UC tests accurately reflect user-driven workflows, including alternative rejection branches and multi-step operational chains. They proved exceptional at catching contract and pagination mismatches between clients and APIs.

3. Business Rules (BR) deliver deep boundary condition probing: While exercising fewer auxiliary endpoints, BR tests focused intensively on numerical thresholds (> Rs. 100,000, > Rs. 500,000), state-transition guards, and boolean validation flags, providing deep safety verification for financial and administrative policies.

4. Workflows (WF) validate state-progression integrity: WF tests traced complete end-to-end lifecycle chains from initial draft to terminal closed states, validating that intermediate audit logs, milestone inspection records, and status locks function harmoniously across prolonged transitions.

6. Candidate Failures & Defect Analysis

In accordance with Section 11 of the Assignment #8 guidelines, every test failure encountered during initial test generation was systematically audited against the four-point confirmation rule:

Issue Description	Affected Strategy	Root Cause Classification	Final Disposition
List Endpoint Pagination: Test asserted list response as a raw list, whereas DRF default PageNumberPagination returns a dict {'results': [...]}	FR (TC_FR_03), UC (TC_UC_09, 11)	Test-Generation / Test-Implementation Problem	Resolved via Prompt C repair (unpacked 'results'). Production code compliant.
Serializer Field Exposure: Test asserted requester_username in create response, but serializer intentionally outputs persisted ID and core status.	BR (TC_BR_01)	Test-Generation / Test-Implementation Problem	Resolved via Prompt C repair (retrieved field from DB entity). Production code compliant.
Confirmed Implementation Defects: Zero discrepancies found between specifications and code.	**FR, UC, BR, WF**	**None (Production code fully conforms to specs)**	**Confirmed Implementation Defects: 0**

7. Repository Deliverables & Verification Index

All experimental artifacts, test suites, execution logs, and data workbooks have been permanently committed to the Git repository on branch `iwd` (commit `7c463a4`) and synchronized with Pull Request #452 on the upstream `FusionIIIT/Fusion-Integrated` repository:

Deliverable Category	Repository-Relative Path	Description & Contents
Frozen Test Designs	a8_design/FR_DESIGN.json a8_design/UC_DESIGN.json a8_design/BR_DESIGN.json a8_design/WF_DESIGN.json	Pre-code serialized JSON arrays defining test_id, source_id, purpose, inputs, and expected outcomes.
Executable Test Suites	tests/a8/fr/test_fr.py tests/a8/uc/test_uc.py tests/a8/br/test_br.py tests/a8/wf/test_wf.py	Automated backend pytest-django test suites executable against isolated test databases.
Machine Results & Logs	a8_results/<strategy>/junit.xml a8_results/<strategy>/coverage.xml a8_results/<strategy>/pytest.log	Machine-generated execution evidence, test durations, and statement/branch coverage metrics.
AI Interaction Logs	a8_results/<strategy>/session_prompt_log.txt	Documented Prompt A, Prompt B, and permitted repair Prompt C text for each strategy session.
Excel Data Collection	Assignment_8_Data_Collection_v2.xlsx	Completed pair-level Excel workbook with Pair & Controls, Strategy Summary, and Artifact Index.
Git Commit & PR	Commit 7c463a4 on branch iwd PR #452 (FusionIIIT/Fusion-Integrated)	Upstream synchronized pull request containing all Assignment 7 implementation and Assignment 8 tests.

8. Pair-Level Compliance Confirmation

We hereby confirm that this study adhered completely to the controlled experimental guidelines specified for Assignment #8:

[X] Production code remained strictly frozen (zero changes to modules/iwd/ or settings during Assignment #8).

[X] The same AI model (Claude Sonnet 4.6 via Antigravity runtime) was used consistently across FR, UC, BR, and WF.

[X] A fresh, independent AI session was initialized for each strategy to eliminate cross-specification memory leakage.

[X] Test designs were formally generated and frozen in JSON format before implementation code was inspected.

[X] The existing 46 Assignment #7 tests in modules/iwd/tests/ were hidden from the AI during test design and generation.

[X] Exact template prompts (Prompts A, B, and C) were utilized without manual editorial embellishment.

[X] The permitted repair budget was strictly respected (FR: 1, UC: 1, BR: 1, WF: 0 — all ≤ 2).

[X] All test expectations were derived directly from specifications and never altered to fit implementation quirks.

[X] All machine-generated evidence files (JUnit XML, coverage XML, pytest logs) have been preserved.

Submitted by:
Meet Patel (Roll No. 24BCS186) & Ashmi Patidar (Roll No. 24BCS282)
PDPM IIITDM Jabalpur — Date: November 2, 2026